Class Activity :

To prepare a Case Study on I/O streams and Library functions in C++

⇒ Case Study: I/O streams and Library functions in C++

1. Introduction

→ I/O streams in C++ provide a powerful and flexible mechanism for handling input and output operations. They abstract the underlying hardware and operating system details, making it easier for developers to work with files, devices and other sources of data. Library functions complement I/O streams by providing additional functionalities and utilities for file manipulation and data processing.

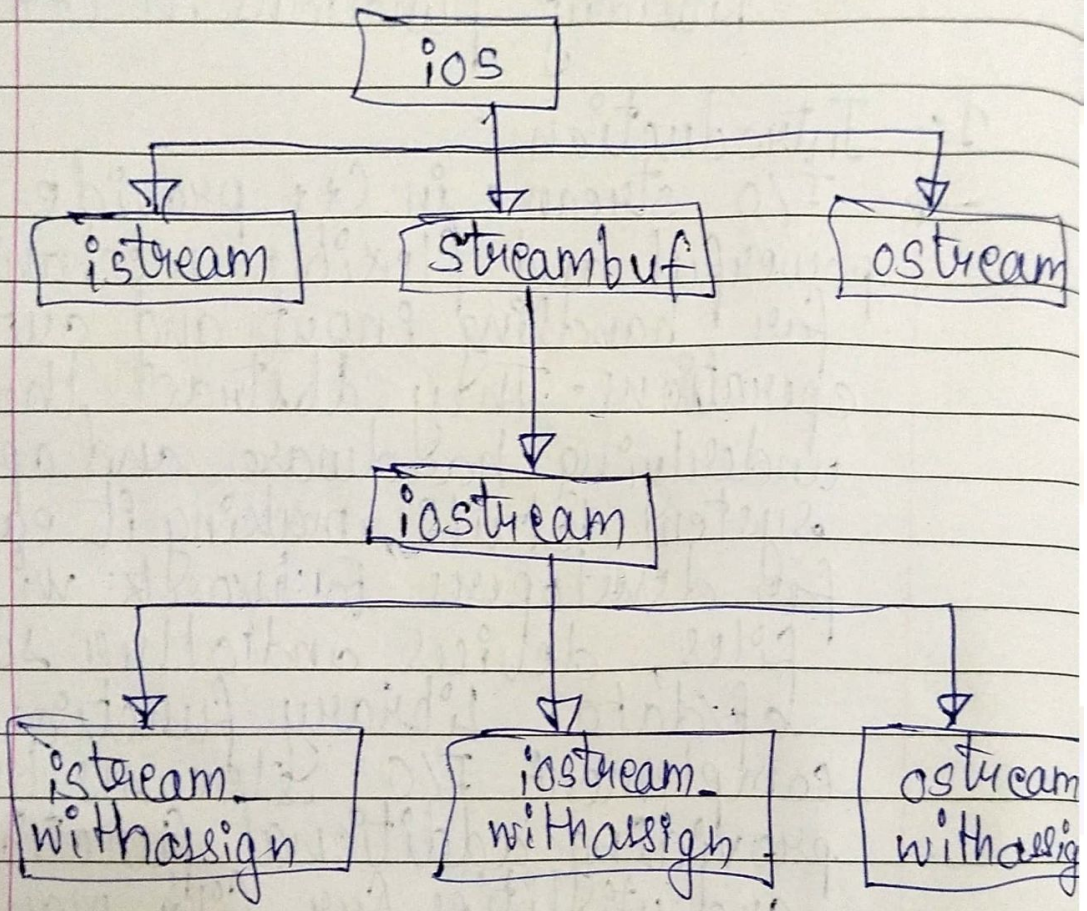
In C++ there are number of stream classes for defining various streams related with files and for doing input-output operations. All these classes are defined in the file iostream.h

PAGE NO.:
DATE: / /

Hierarchy of Stream

2] Diagramatic Representation:

Hierarchy of stream classes in iostream.h



3] Description of I/O Files / Streams

- ifstream: Used for reading data from files.
- ofstream: Used for writing data to files.
- fstream: Used for both reading and writing data to files.

- cin : Standard input stream, typically associated with the st keyboard.
- cout : Standard output stream, typically associated with the console.
- cerr : Standard error stream, used for error messages.
- clog : Standard logging stream, used for logging information.

4] Examples and Syntax

- Reading from a file using ifstream

```
#include <iostream>
#include <fstream>
int main()
{
    std::ifstream infile("input.txt");
    if (infile.is_open())
    {
        std::string line;
        while (getline(infile, line))
        {
            std::cout << line << std::endl;
        }
```

```
    infile.close();
}
else
{
    std::cout << "Error
        opening file" << std::
                                endl;
}
return 0;
}
```

- Writing to a file using ofstream:

```
#include <iostream>
#include <fstream>
int main()
{
    std::ofstream outfile.("output.
        txt");
    if (outfile.is_open())
    {
        outfile << "Hello world!"
            << std::endl;
        outfile.close();
    }
    else
    {
        std::cout << "Error opening
            file" << std::endl;
```

```
}
return 0;
}
```

5] Library functions with Example

→ open(): Opens a file
→ close(): closes a file
→ seekg(): Sets the input file pointer to a specific position.
→ tellg(): Returns the current position of the input file pointer.
→ seekp(): Sets the output file pointer to a specific position
→ tellp(): Returns the current position of the output file pointer
→ eof(): checks if the end of the file has been reached.

Example using seekg and tellg:

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream infile("input.txt");
```

```
if (infile.is_open())
  {
    infile.seekg(10);
    std::cout << "Current
position: " << infile.tellg()
     << std::endl;
    infile.close();
  }
else
  {
    std::cerr << "Error
opening file:" << std::endl;
  }
  return 0;
}
```

Conclusion: I/O streams and library functions in C++ provide a powerful and flexible framework for handling file and data operations. By understanding the concepts and syntax, developers can efficiently read, write, and manipulate data in various formats.